# Evaluate a Real-Time Multi-Turn Chatbot


## Abstract

Companies are transitioning from traditional chatbots and human agents to multi-agent systems powered by large language models (LLMs) with RAG to answer business questions and even call APIs directly or through MCP. The core risk of AI products is: you cannot scale what you cannot measure. AI quality is hard to prove because any change to prompts, models, or logic is risky. Any of these failures directly hit cost, trust, and retention. We did research on evaluation methods on a real-world application dataset and developed an evaluation platform to test several common scenarios for industry applications.


## 1. Introduction

LLM application evaluation is critical during development and after launch. Based on different user scenarios, we need to create real-world datasets, benchmarks, metrics, evaluation methods, and result visualization. This paper discusses how to evaluate a real-time multi-turn conversation chatbot.

## 2. Methodology

### 2.1 Dataset

#### 2.1.1 Data Pipelines

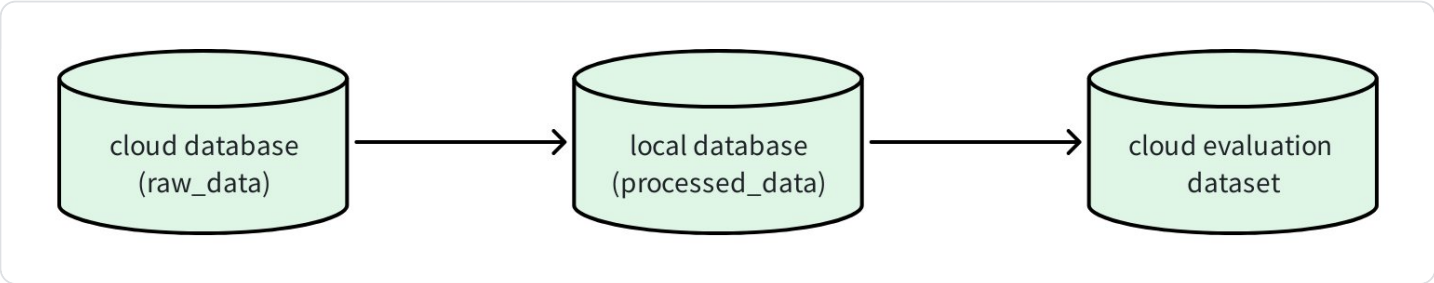
The most useful data is real queries from customers. If your production is not deployed yet, we suggest you deploy a simple chatbot even if it is just an end-to-end model or simple RAG QA bot. Once you have real queries from customers, you can analyze the category of query, number of conversation rounds, latency of chatbot, etc. All of these analysis results can be used as benchmarks to evaluate the AI chatbot.

When you deal with raw data, the centralized data pipeline is:

a. Crawl raw data from database

b. Process data to usable format

c. Upload the data with only files that are going to be evaluated on




### 2.1.2 Dataset Format

We keep data columns as *id, inputs, reference_output* as required, and all other columns can be customized. We suggest types of inputs as follows:

- **Single-turn:** One query, one answer.

| id | inputs | reference_output |
|---|---|---|
| 001 | ["hi, who are you?"] | ["GPT"] |

- **Multi-turn (Format 1):** Multiple queries per row, each row (conversation) evaluated independently.

| id | inputs | reference_output |
|---|---|---|
| 001 | ["hi", "who are you?"] | ["GPT"] |

- **Multi-turn (Format 2):** One query per row, evaluated as conversation.

| id | inputs | reference_output |
|---|---|---|
| 001 | ["hi"] | ["hello"] |
| 002 | ["who are you?"] | ["GPT"] |

### 2.1.3 Synthetic Dataset

If you are unable to secure access or have any real data, you can start off with some synthetic datasets for benchmarks.

For our support bot, we created the synthetic data benchmarks by going over all our support content (articles, FAQs, YouTube videos, etc.) and came up with user queries in a backward approach from the answers.

In the case of a support bot, this can also be used to evaluate your RAG search.

But keep in mind that you will need real data at some point in the future, maybe by using user data after the launch of your chatbot.

If you are evaluating other agents than a support bot, something that can create something for you (a data object, text, etc.), you can generate this data using LLMs and then use them appropriately. Please note that this would require proper prompt engineering or tuning your LLM.

Another thing to pay attention to in synthetic data generation is the cost. If you are using your own GPU clusters to run open-source LLMs, it can be cheaper. But if you are using APIs from OpenAI, Anthropic, etc., depending on the scale of synthetic data that you want to create, the costs can increase quickly.

## 2.2 Metrics

The proper metric is helpful in evaluation. You cannot use only one metric for different dimensions of evaluation. We try to detect failures before the customer does, and we can compare versions safely (regression testing). We will discuss different metrics based on the method we are using.

1. **LLM as Judge**
    a. **Correctness** -- Does the answer match the expected output in intent, scope, and detail?
    b. **Relevance** -- Is the response relevant to the user's question?
    c. **Comparison** -- Pairwise comparison with an existing bot (third party or prior versions). Pay attention to the positioning of the two responses you are trying to compare. You may want to evaluate them independently and just compare the scores. Or if you want to evaluate them by using an llm in one API call, then we strongly suggest randomizing their positions to mitigate any potential position bias that are common in LLMs.
    d. **Hallucination** -- Requires a source of truth to detect fabricated information
    e. **Agent Match** -- In a multi-agent system with an orchestrator to handle routing, does the correct agent handle the query?
2. **Code-Based Evaluation**
    a. **Fields Alignment** -- In the case of agents that perform tasks like object creation (JSON, Python dict, etc.), the evaluator can be deterministic for fields that require an absolute match, or can be a semantic match, or you can have another LLM as judge.
3. **Efficiency**
    a. Latency
    b. Token usage
    c. Cost

4. **User Satisfaction**
    a. Number of turns of conversation to resolve a specific issue or complete a task from end to end.
5. **Human Judge (If you can afford it :P)**
    a. Manual review and annotation for ground truth validation.

## 2.3 Tools

Evaluation platform is built in Python and displays results in real-time dashboards via LangSmith. As long as you are using LangChain or LangGraph in your system, it is easy to integrate results into LangSmith.

Based on your organization size, bandwidth, and situation, you can use other third-party platforms or in-house platforms for the same purpose. You just need a dashboard where you can see the results.

| Name | Input | Output | Error | Start Time | Latency | Dataset | Annotation Queue | Tokens | Cost |
|---|---|---|---|---|---|---|---|---|---|
| LangGraph | human: Hi when is my next appointment? | ai: Hi, you're talki... | | 1/9/2026, 3:18:47 PM | 4.03s | | | 3,635 | $0.0098 |
| LangGraph | human: Yes | ai: Thank you for ... | | 1/9/2026, 3:17:54 PM | 1.60s | | | 960 | $0.003105 |
| LangGraph | human: Hi I have an appointment at 4 today th... | ai: Hi, you're talki... | | 1/9/2026, 3:17:36 PM | 3.16s | | | 1,190 | $0.0037025 |
| LangGraph | human: Thank you! | ai: Hi, you're talki... | | 1/9/2026, 2:48:06 PM | 1.98s | | | 825 | $0.0023625 |
| LangGraph | human: I submitted my n nomination. Good luck! | ai: Hi, you're talki... | | 1/9/2026, 2:28:53 PM | 2.03s | | | 631 | $0.0019225 |
| LangGraph | human: I don't be need anything this week but... | ai: Hi, you're talki... | | 1/9/2026, 1:59:30 PM | 1.50s | | | 1,374 | $0.00378 |

# 3. Experiments

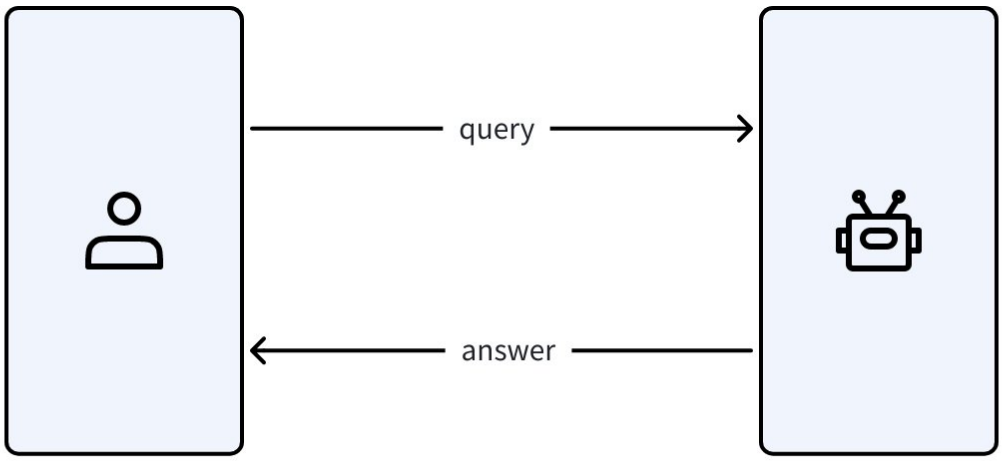
## 3.1 Experimental Setup

We use CSV and JSON format to keep track of data and load it to the evaluation platform. We connect to the system under test (SUT) via API with webhook and Python script. We created a few evaluators locally from different dimensions to evaluate on the dataset.

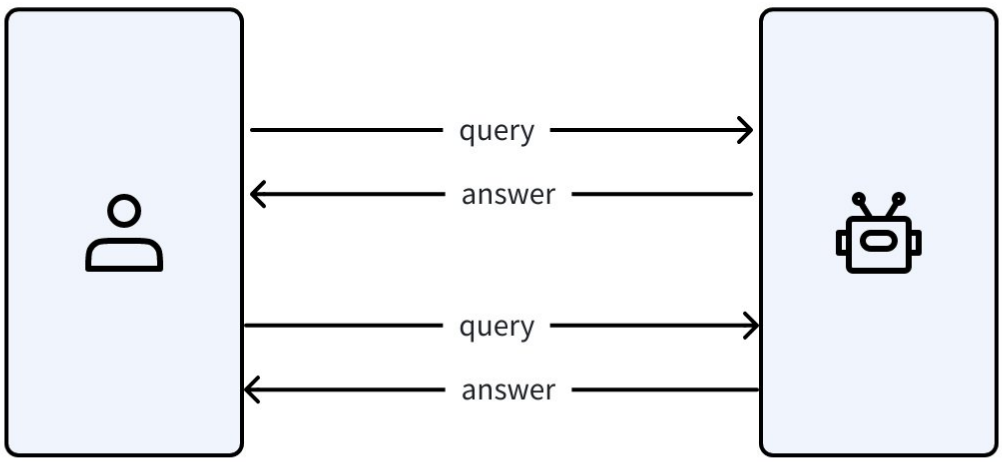
Because we are evaluating a chatbot, we set up two conversation methods: "single turn" and "multiple turns". A single turn can test a clean environment without any context--how the chatbot responds. Multiple turns can simulate user interaction with the chatbot and how context affects the chatbot.

For multiple turns, we set up two experiments:

1. **Multiple turns saved in one piece of data**: We evaluate every single turn.
2. **Multiple data in one dataset**: We only evaluate the last turn in each single data, but we have multiple evaluations in one dataset. Therefore, we save queries in a list instead of string. It is easy to reuse.




Single-turn

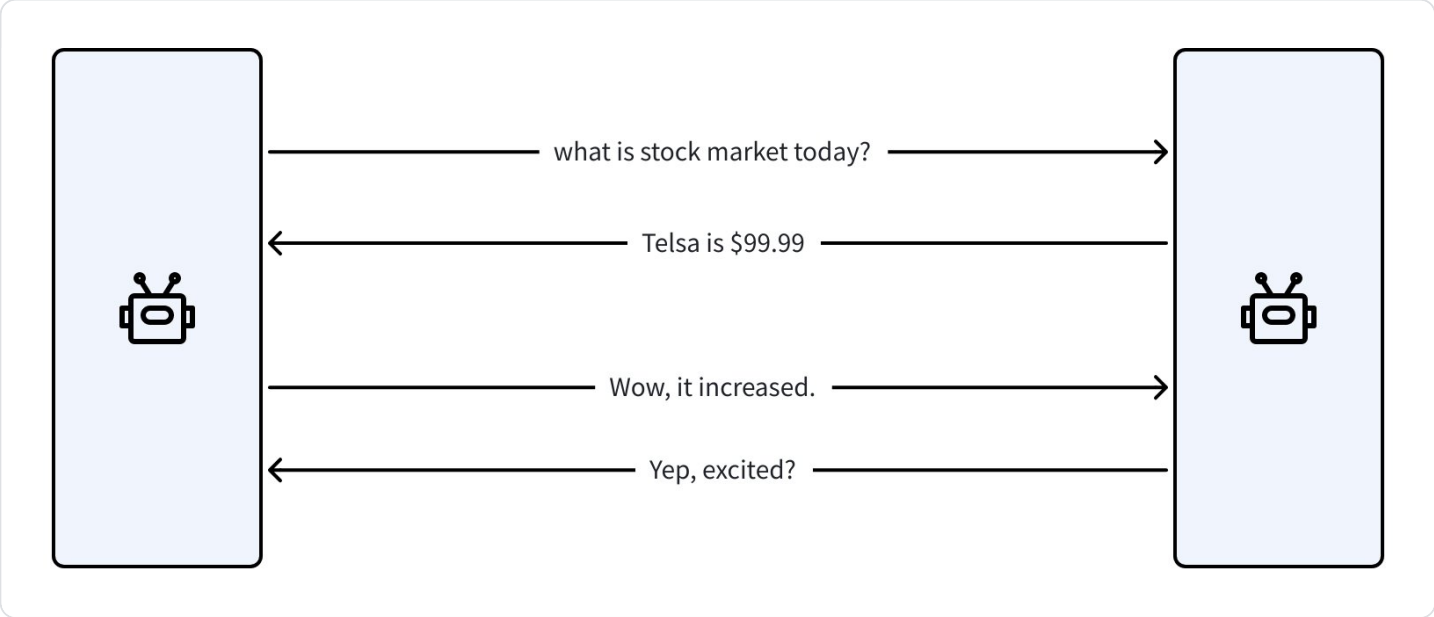



multi-turn

## 3.2 AI Provider Evaluation

Since the chatbot is driven by LLMs, we cannot predict the response from the bot. The predefined query sometimes is not the real answer or query to the bot. Therefore, we create an automated evaluation pipeline simulating a real user by using another LLM to respond to the chatbot based on the chat history.

Instead of using a static, predefined dataset, this simulator uses a secondary AI to generate dynamic user queries in real-time based on chatbot responses. Randomized topic injection (such as pricing, technical support, and service inquiries) dynamically generates follow-up queries based on the SUT's live responses.




We know real-world chatbots allow users to chat with AI and then call APIs to replace engineering calling. No matter if the chatbot is connecting to APIs directly or through MCPs, the chatbot will return a payload before calling APIs, so we will evaluate the payload of the API call, which is a JSON file.

If you have a data benchmark (real or synthetic) and, as mentioned earlier, due to the non-deterministic nature of LLMs, you cannot expect the conversation to go in the same direction as your benchmark. For example, if your data benchmark looks like this:

- user_query = ["what's the stock market like for Tesla today?", "Wow, it increased."]
- reference_output = ["Tesla is $99.99", "Yep, excited"]

But when you are running this eval, the LLM outputs something like, "oh... Tesla is down by 23% today" for the first query. In this case, it wouldn't even make any sense to ask the 2nd user_query, right?

To handle these situations, we need a more sophisticated evaluation system--something that can change the follow-up query based on the response to the previous response by the bot. We need intelligence from both sides.

This is why we came up with the approach of having an agent which simulates a user which then talks to the bot. But other than just simulating the different categories / types of users that you have (price evaluator, early adopters, non-tech users etc.), we need to have an objective for this user simulator agent. You need to know what the goal of the conversation is. What exactly is the objective or end-goal that it is trying to accomplish with this conversation with the bot (support or other AI agent).

For that we came up with 2 approaches:

1. **LLM-based conversation analysis**: Have a customized LLM (either fine-tuned or prompt engineered) analyze each conversation in the benchmark and figure out what problem the user was trying to solve through it. Then hand over the structured problem (with multiple sub-problems when necessary) to the user simulator agent. This requires careful creation and tuning. Now you can use this user simulator agent to talk with your bot and see how long it takes to accomplish the objective (or whether it's even possible to accomplish it).
2. **Data-driven topic distribution**: Another approach is to do some data analysis on your real user data. Most of the time, you'll find there is actually a very good distribution of problems and concerns. It is highly likely that there are only a few categories or topics that lead to the majority of the questions. You can just give these topics to the user simulator agent and tune it to ask questions just based on topic and give it free reign on that.

There are different trade-offs to each approach. You can think of your conditions, your data, your organization's time and cost limits, etc., to decide what works best for your particular case.

One thing to keep in mind about these approaches is the cost. If you are a small company or a startup and are using API providers for LLM calls, this could raise your costs significantly for every eval that you run.

### 3.2.1 Time

We discussed the cost in terms of dollars, but another thing to keep in mind is the time it takes to evaluate a full data benchmark. If you are doing it sequentially, then the time could take from 2-3 hours to even days depending on the size of your dataset.

So we decided to parallelize the process. The obvious advantage is that you can run evals much faster, but another benefit is that you will be stress testing your system for real usage. We need to analyze and predict expected load once the bots are launched and can simulate the same when running our evals. This helped us significantly in testing both system reliability and availability along with speeding up every run of the eval. The benefit of which is felt in the long run when you start some internal user testing and you need to iterate quickly and make improvements, especially like prompt engineering or hyper-parameter tuning.

## 3.3 Results

There is no one common evaluator to evaluate all different data. Evaluators need to be designed for different benchmarks, so it is important to develop a platform to plug in new evaluators quickly for your product to support more and more complex production requirements as they come out. Basic evaluators can be set for most benchmarks like correctness.

AI chatbot response is difficult to predict, and in tons of cases, AI-to-AI methods with LLM-as-judge are very important and useful to test a real-time bot.

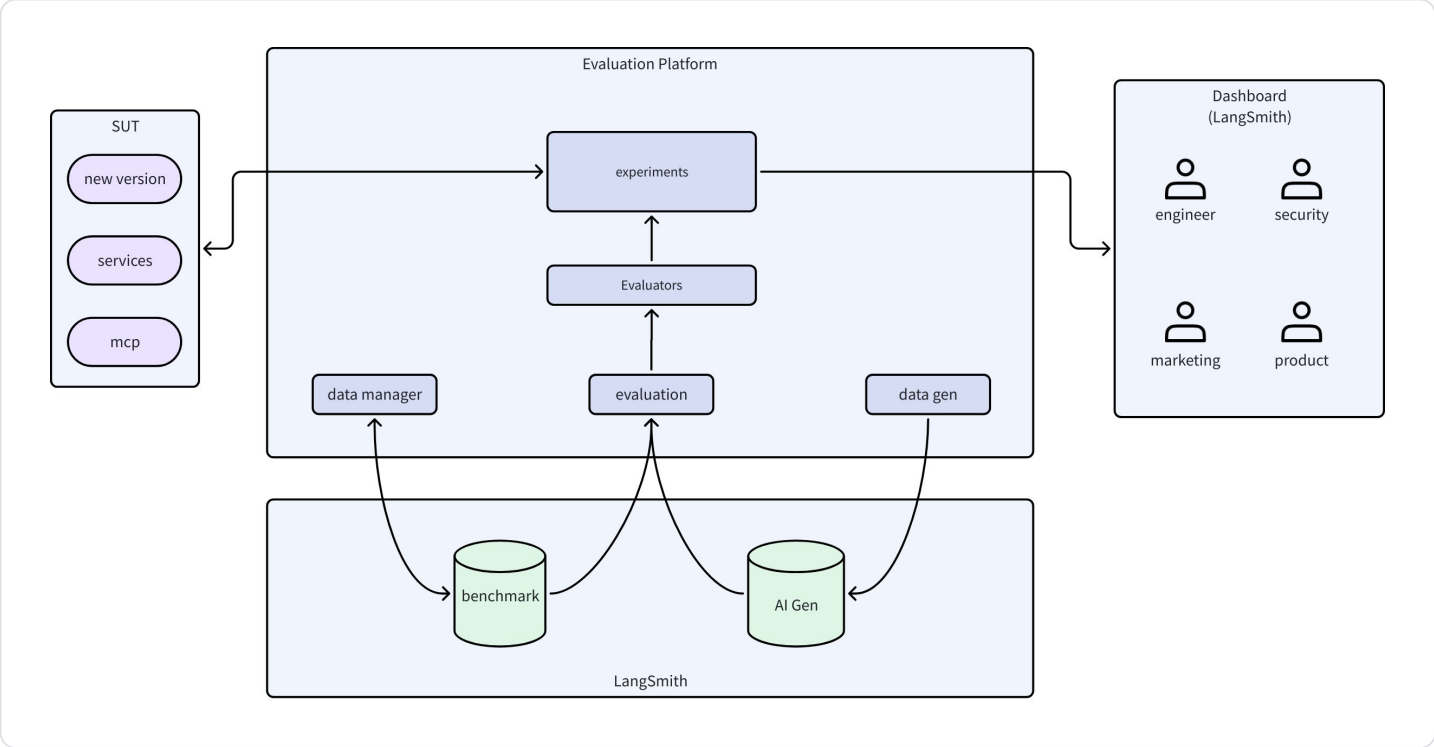
We want to get rid of humans in the loop as much as possible, so once evaluation is done, collecting all bad cases together and rethinking how to provide feedback to improve the SUT system is very important. It will be chatbot reinforcement learning process.

| Agent_match 1.00 AVG | Correctness 0.429 AVG | Relevance 1.00 AVG | Latency 10.13 P50 |
|---|---|---|---|
| 1.00 | 1.00 | 1.00 | 9.04s |
| 1.00 | 0.00 | 1.00 | 5.33s |
| 1.00 | 0.00 | 1.00 | 12.97s |
| 1.00 | 1.00 | 1.00 | 22.71s |
| 1.00 | 0.00 | 1.00 | 26.95s |
| 1.00 | 0.00 | 1.00 | 10.13s |
| 1.00 | 1.00 | 1.00 | 8.01s |

# 4. Conclusion

We work on the evaluation of real-time chatbots. The evaluation can help the SUT get better and better. Therefore, evaluating intent alignment, details coverage, constraint satisfaction, link consistency, misleading or incorrect guidance are all important metrics.

## 4.1 System Architecure




### 4.1.1 Evaluation Pipeline

- SUT (new version, services, MCP) connects to the evaluation platform
- Client manager handles single-turn and multi-turn modes
- Evaluation provider feeds into LLM-as-judge evaluators
- LangSmith provider for data storage and experiment tracking
- Model manager for LLM configuration

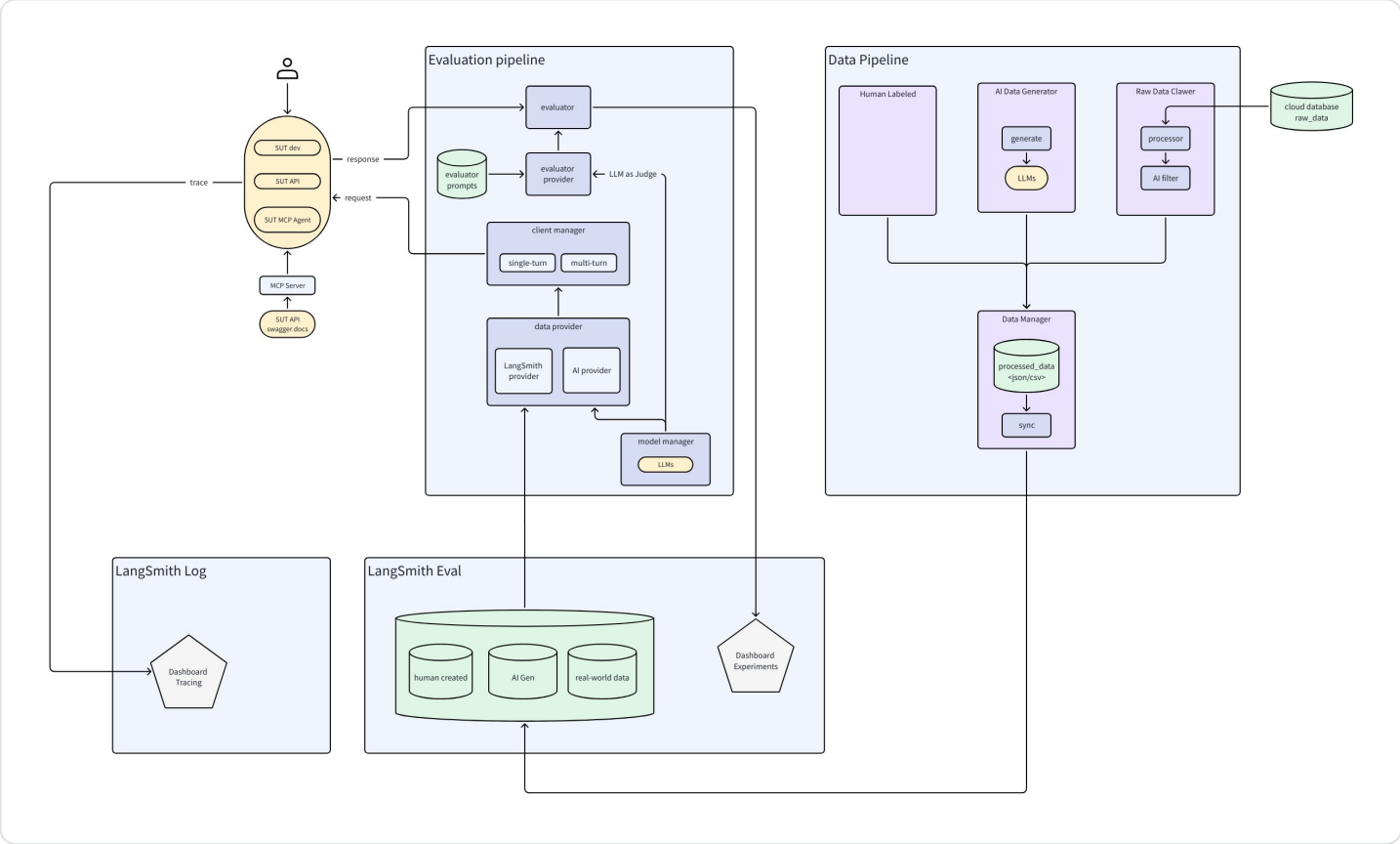
### 4.1.2 Data Pipeline

- Human-labeled data and AI data generators feed into the data pipeline
- Client-server chat data flows through processing and filtering
- Processed data and AI-generated data merge into the evaluation dataset

### 4.1.3 LangSmith Integration

- Dashboard and tracing for LangSmith logs
- Human-created, AI-generated, and real-world data stored in LangSmith evaluation
- Dashboards and experiments visible to engineers, security, marketing, and product teams

## 4.2 The Full Picture




# 5. Reference


1. Demystifying evals for AI agents
2. Evaluating Multi-Turn Conversations
3. Evaluate Multi-Turn Convos
4. Evaluating Multi-turn Human-AI Conversations
5. Evaluating Multi-Turn LLM Chat Conversations Using Label Studio


# 6. Contact Information


- Stephen Wang
  - Email: zhongqi1112@gmail.com
  - Website:https://stephengineer.github.io
- Lohith Kumar Bhambore
  - Email: lohithkumar24@duck.com
  - Github: https://github.com/EmileS24